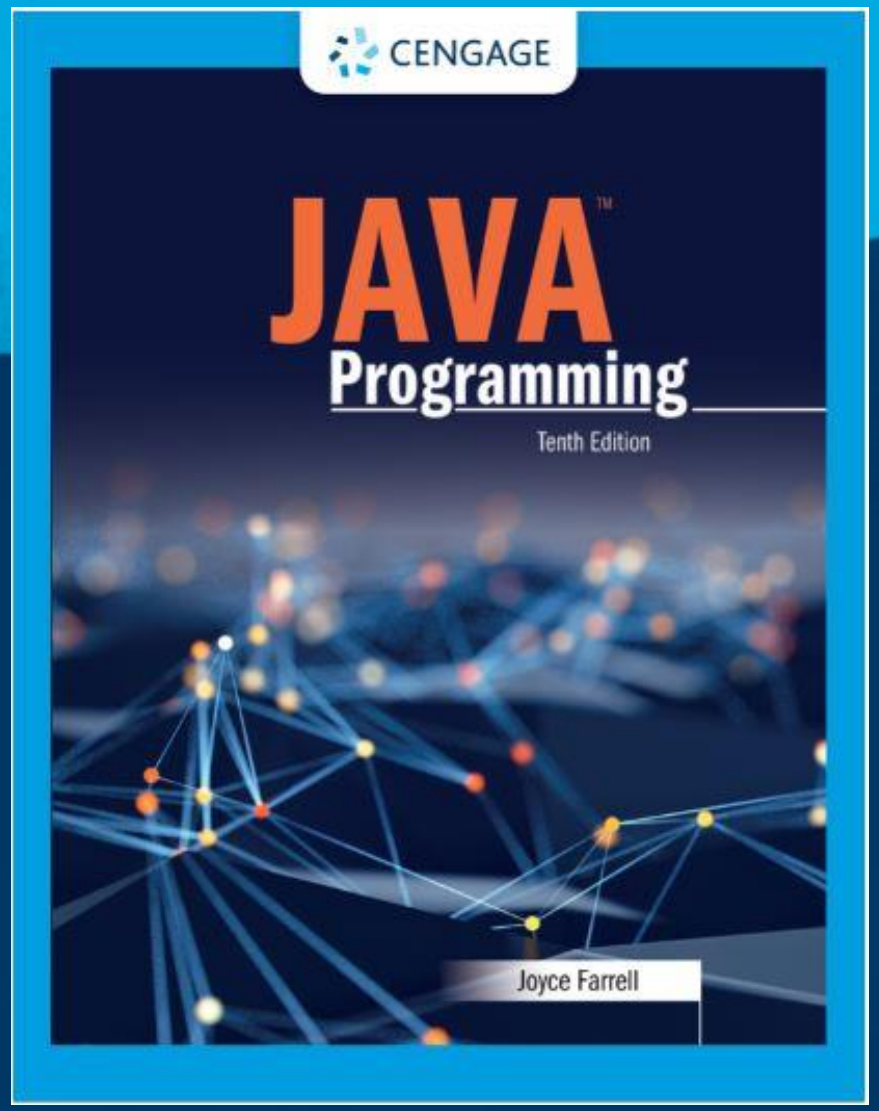



# Java Programming, 11e

## Chapter 11: File Input and Output

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# Chapter Objectives

By the end of this chapter, you should be able to:

11.1 Describe computer files

11.2 Use the `Path` and `Files` classes

11.3 Describe file organization, streams, and buffers

11.4 Use Java's IO classes to write to and read from a file

11.5 Create and Use sequential data files

11.6 Describe random access files

11.7 Write records to a random access data file

11.8 Read records from a random access data file

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.1 Understanding Computer Files (1 of 6)

## Volatile storage

- Temporary
- Lost when computer loses power
- RAM (random access memory)
- Java programs that store a value in a variable use ram

## Nonvolatile storage

- Permanent
- Java programs saved to disk use permanent storage

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.1 Understanding Computer Files (2 of 6)

## Computer file

- Collection of data stored on a nonvolatile device
- Exists on permanent data storage device
  - Hard drive
  - USB drive
  - Magnetic tape
  - Compact disc

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.1 Understanding Computer Files (3 of 6)

## Text file

- Data that can be read in a text editor
- Coded with ASCII or Unicode
  - Data file
  - Program or application file

## Binary file

- Data not encoded as text
- Images, music, and compiled program files with *.class* extension


Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.1 Understanding Computer Files (4 of 6)

## Root directory

- Main directory of storage device
- Folders and directories used to organize files
- A path is a file's list of hard drive plus hierarchy of directories
  - Path delimiter: a \ that separates path components

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.1 Understanding Computer Files (5 of 6)

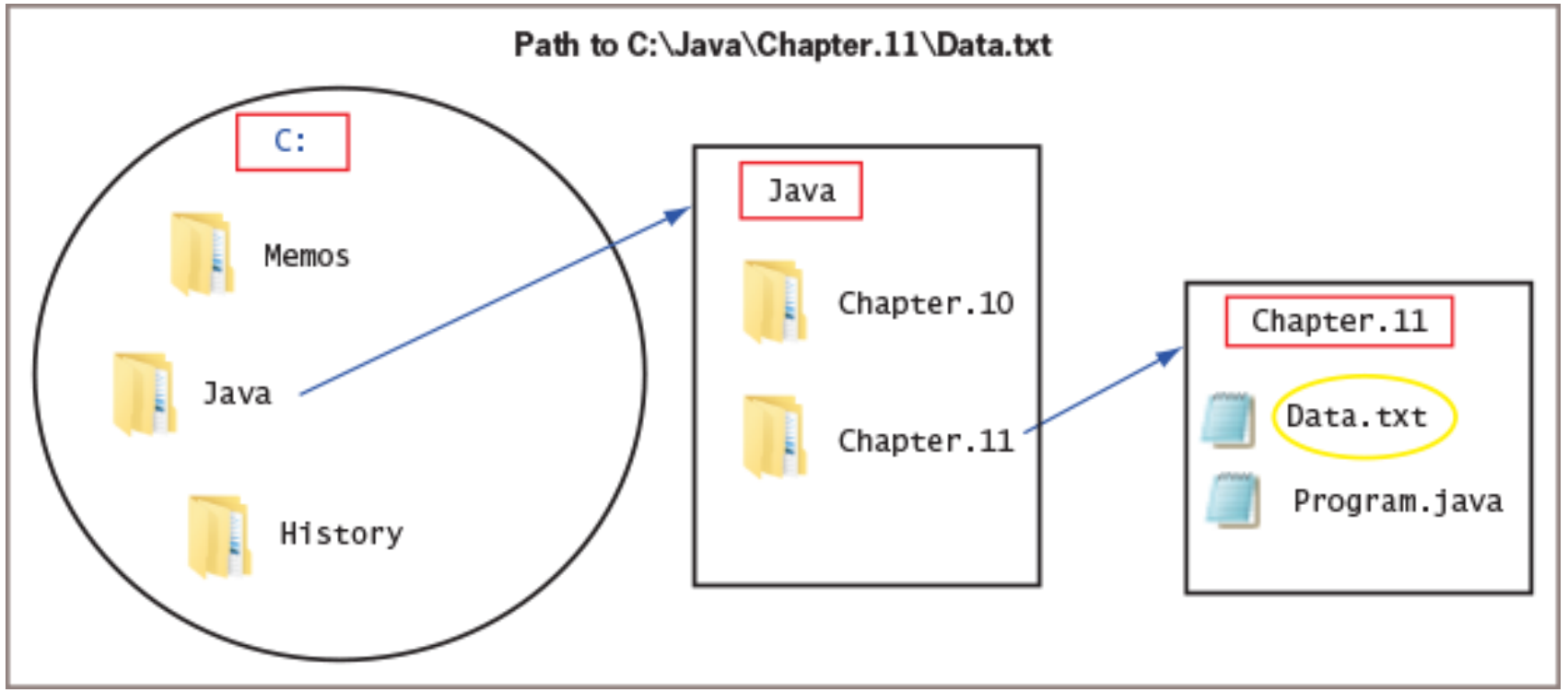



Figure 11-1: Location of *C:\Java\Chapter.11\Data.txt*

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.1 Understanding Computer Files (6 of 6)

## Typical tasks performed with files

- Determining whether and where a path or file exists
- Opening a file
- Writing to a file
- Reading from a file
- Closing a file
- Deleting a file

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.2 Using the `Path` and `Files` Classes (1 of 6)

## `Path` class

- Used to create objects that contain information about files and directories
- Locations, sizes, create dates, and whether they exist

## `Files` class

- Used to perform operations on files and directories
- Deleting, determining attributes, and creating input and output streams


Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.2 Using the `Path` and `Files` Classes (2 of 6)

## Creating a `Path`

- First determine the default file system on the host computer
  - `FileSystem fs = FileSystems.getDefault();`
  - Creates a `FileSystem` object
- Define a `Path` using `getPath()` method
- Use `Path` identifier to refer to the file and perform operations

## `Path`

- Absolute: a complete path
- Relative: depends on other path information

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.2 Using the `Path` and `Files` Classes (3 of 6)

## Retrieving `Path` information

- `String toString()`
  - Returns `String` representation, eliminating double backsplashes
- `Path getFileName()`
  - Returns file or directory denoted by this `Path`
- `int getNameCount()`
  - Returns number of name elements in the `Path`
- `Path getName(int)`
  - Returns name in the position of the `Path` specified by the integer parameter

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.2 Using the `Path` and `Files` Classes (4 of 6)

## Convert a relative path to absolute

- `toAbsolutePath()`
- If input represents relative `Path`, an absolute path is created
- Assigns the file to the current directory

## Check file accessibility

- `checkAccess()`
- No argument checks the file exists
- `READ`, `WRITE`, `EXECUTE`
  - Check the file exists and if permission exists for action
- `static import` feature

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.2 Using the `Path` and `Files` Classes (5 of 6)

## Delete a `Path`

- `delete()`
- Cannot delete a file that doesn't exist
- Directory cannot be deleted unless empty
- `SecurityException` thrown if don't have permission to delete
- Input/output errors cause an `IOException`

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.2 Using the `Path` and `Files` Classes (6 of 6)

## Determine File attributes

- `readAttributes()` method
  - `Path` object and `BasicFileAttributes.class`
- `Size` method
  - Size of file in bytes
- `creationTime()` and `lastModifiedTime()`
  - Important file times

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.3 File Organization, Streams, and Buffers (1 of 4)

## Character

- Smallest useful piece of data to most users
- Letter, number, or symbol that comprises data
- Made up of bits

## Field

- Group of characters with some meaning

## Record

- Collection of fields that contain data about an entity

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.3 File Organization, Streams, and Buffers (2 of 4)

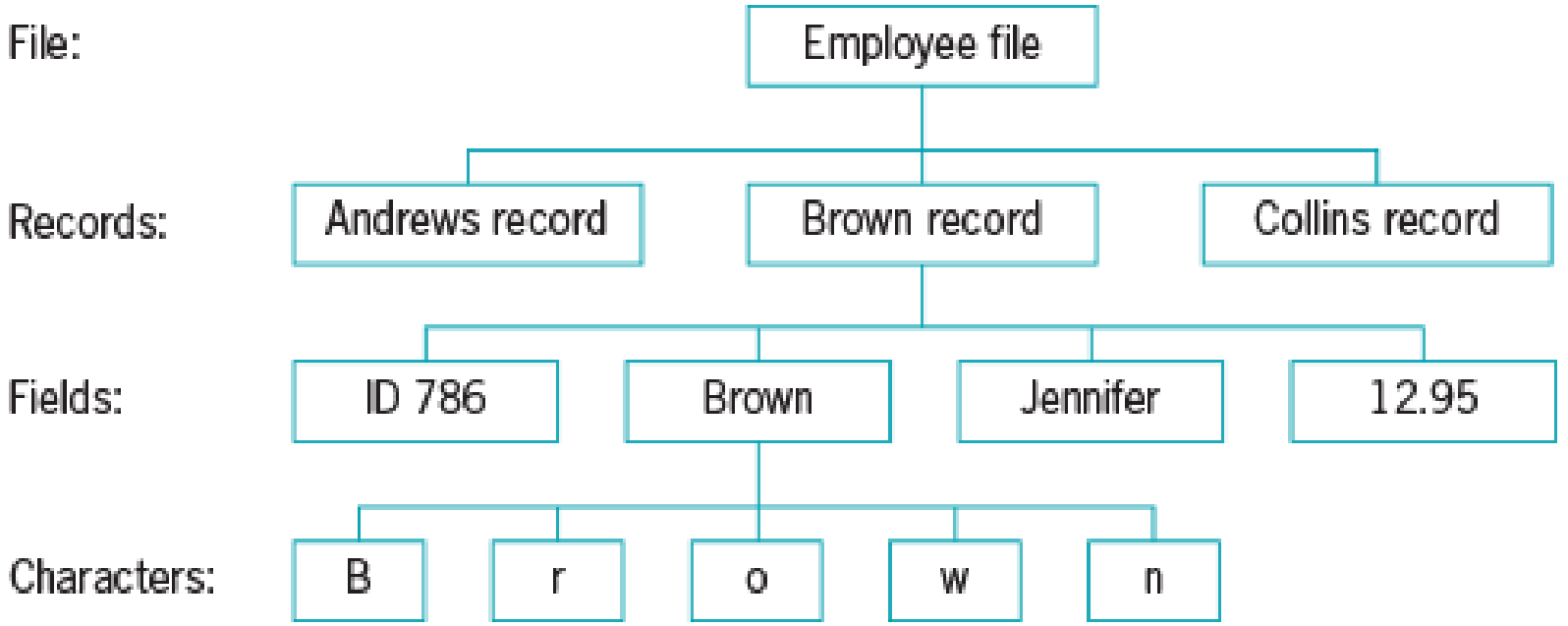



Figure 11-13: Data hierarchy

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.3 File Organization, Streams, and Buffers (3 of 4)

## Sequential access file

- Data files can be used this way
- Each record is accessed in the order in which it was stored

## Comma-separated values (CSV)

- Each value in a record is separated by a comma

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.3 File Organization, Streams, and Buffers (4 of 4)

## Open a file

- Make it accessible

## Close a file

- Make it no longer available to an application

## Stream

- Data arrives via an input stream
- One output stream writes some records to a file of valid records
- Another output stream writes other records to a file of invalid records

## Flushing

- Clears any bytes that have been sent to a butter but not output to device


Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.4 Using Java's IO Classes (1 of 4)

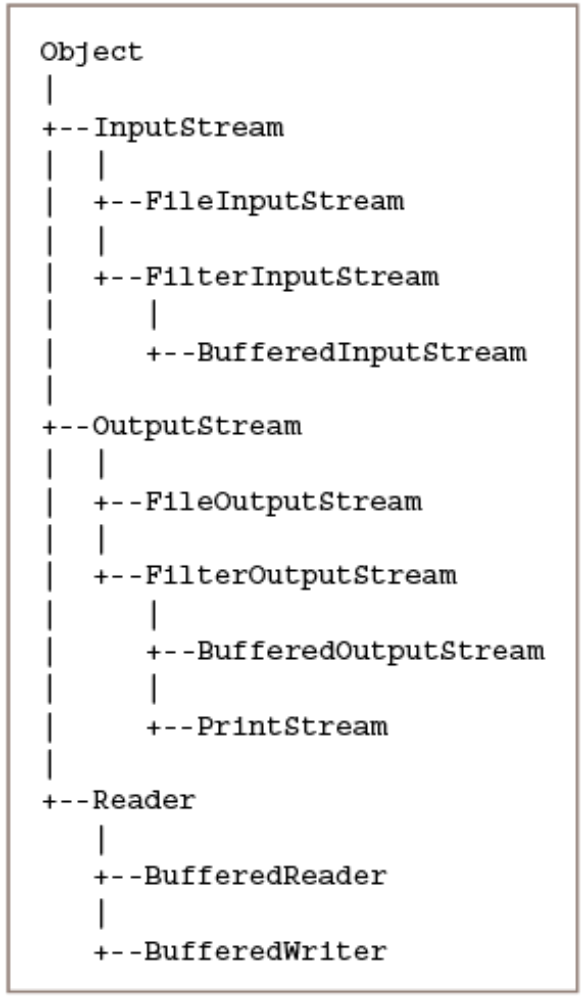



Figure 11-15: Relationship of selected IO classes

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.4 Using Java's IO Classes (2 of 4)

## `InputStream`, `OutputStream`, and `Reader`

- Subclasses of `Object` class
- Abstract
- If necessary you can create your own `OutputStream` object and assign `System.out`

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.4 Using Java's IO Classes (3 of 4)

## Write to a file

- Instead of a standard output device to `OutputStream`
- Assign a file to a `Stream` object
  - Screen output and file output work the same
- Use the `Files` class `newOutputStream()` method
- `WRITE`, `APPEND`, `TRUNCATE_EXISTING`, `CREATE_NEW`, `CREATE`, `DELETE_ON_CLOSE`

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.4 Using Java's IO Classes (4 of 4)

## Read from a file

- `Scanner` class for keyboard input
- `InputStream` class to input data stored in a file
- `BufferedReader` methods
  - `close()`: closes the stream and its resources
  - `read()`: reads a character
  - `readLine()`: reads a line
  - `skip(long n)`: skips specified number of characters

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.5 Creating and Using Sequential Data Files (1 of 3)

## Save more than one `String` to a file

- `BufferedWriter` class
- Counterpart to `BufferedReader`
- Writes text to an output stream, buffering the characters
- `close()`, `flush()`, `newline()`, `write (int c)`

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.5 Creating and Using Sequential Data Files (2 of 3)

```
Enter employee ID number >> 234
Enter name for employee #234 >> Harris
Enter pay rate >> 19.65
Enter next ID number or 999 to quit >> 319
Enter name for employee #319 >> Ravani
Enter pay rate >> 21.15
Enter next ID number or 999 to quit >> 451
Enter name for employee #451 >> Garcia
Enter pay rate >> 25.75
Enter next ID number or 999 to quit >> 999
```

Figure 11-22: Typical execution of the `WriteEmployeeFile` program

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.5 Creating and Using Sequential Data Files (3 of 3)

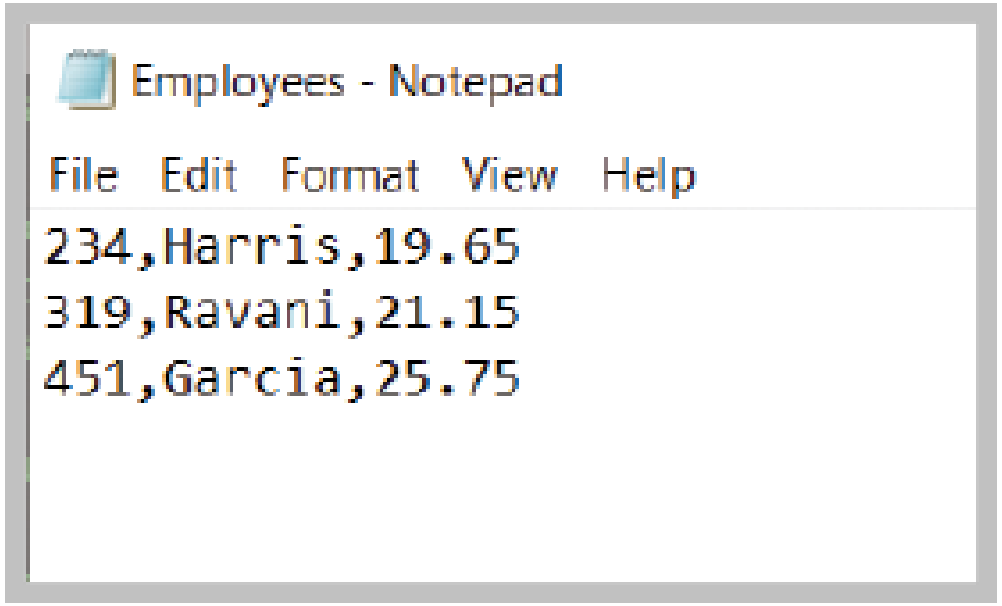



Figure 11-23: Output of the execution of the `WriteEmployeeFile` program

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.6 Learning About Random Access Files (1 of 4)

## Batch processing

- Data stored in sequential order
- Performing the same tasks with many records
- One after another
- Doesn’t matter which is produced first

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.6 Learning About Random Access Files (2 of 4)

## Real-time applications

- When sequential access is inefficient
- Records accessed immediately

## Interactive program

- User makes a direct request

## Random access file

- Records accessed or retrieved in any order
- Also called direct access or instant access


Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.6 Learning About Random Access Files (3 of 4)

## File channel object

- Avenue for reading and writing a file
- `FileChannel` class used to create a random access file

## Wrapped

- An array of bytes encompassed into a `ByteBuffer`
- Holding place for bytes waiting to be read or written

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.6 Learning About Random Access Files (4 of 4)

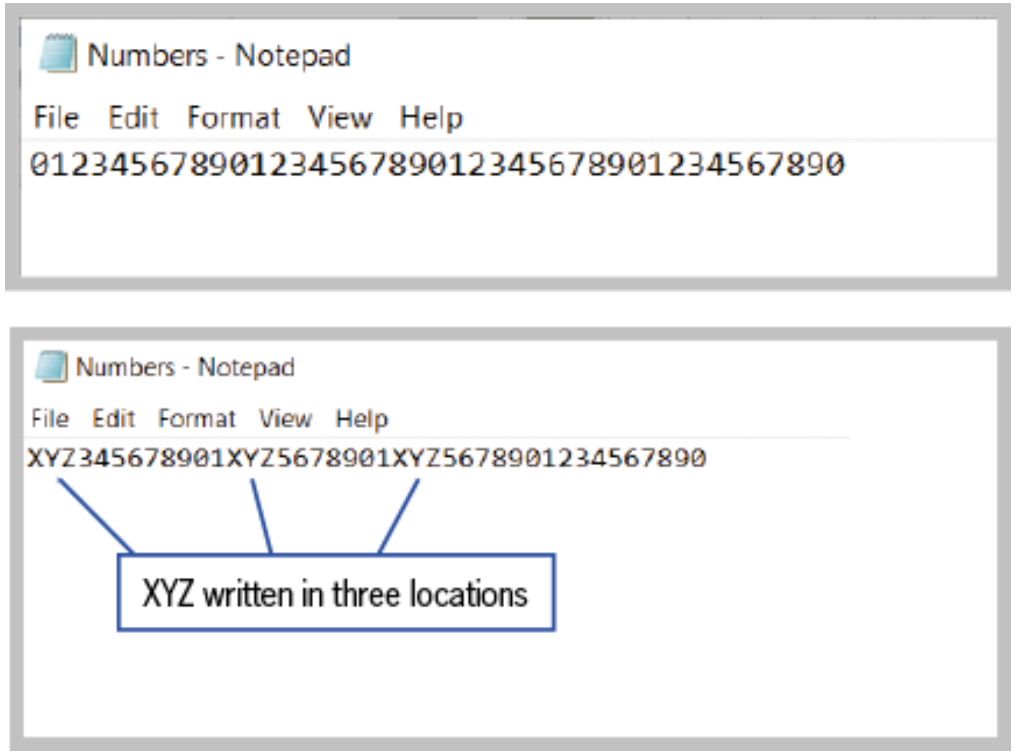



Figure 11-29: The *Numbers.txt* file before and after the execution of `RandomAccessTest` program

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# Activity 11.3: Knowledge Check

A(n) _____ program is one in which the user makes direct requests.

a. sequential

b. front-end

c. volatile

d. interactive

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# Activity 11.3: Knowledge Check Answer

A(n) _____ program is one in which the user makes direct requests.

**Answer: d. interactive**

A program in which the user makes direct requests is an interactive program. For example, if a customer telephones a department store with a question about a monthly bill, the customer service rep requires a random access file, in which records can be accessed or retrieved directly in any order.

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.7 Writing Records to a Random Access Data File (1 of 3)

## Key field

- Field in a record that makes it unique
- Used in writing a random access file
- First, create a file that holds default records
- Then you can replace any records with actual data

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.7 Writing Records to a Random Access Data File (2 of 3)

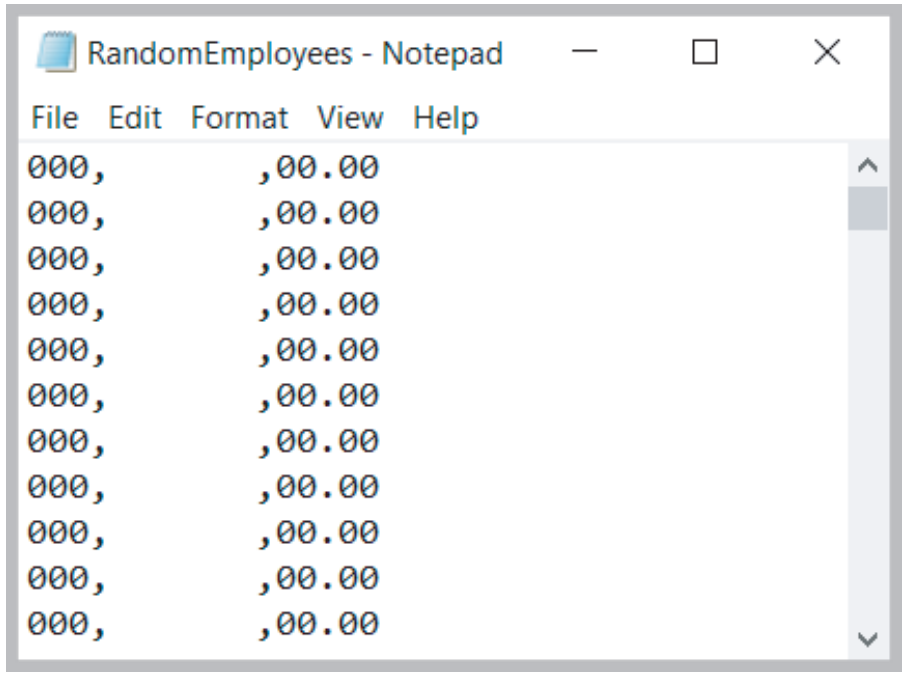



Figure 11-31: The *RandomEmployees.txt* file created by the `CreateEmptyEmployeesFile` program

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.7 Writing Records to a Random Access Data File (3 of 3)

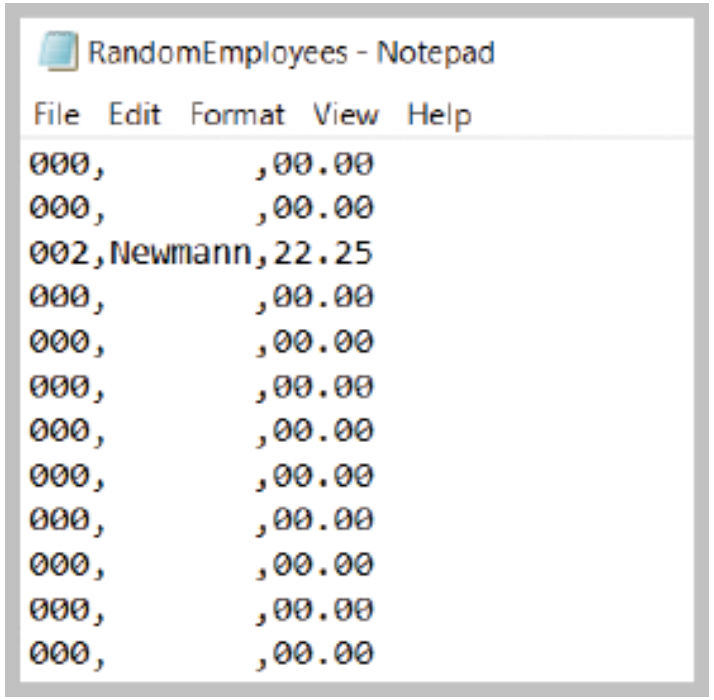



Figure 11-33: The *RandomEmployees.txt* file created by the `CreateOneRandomAccessRecord` program

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.8 Reading Records from a Random Access Data File (1 of 3)

## Accessing a random access file sequentially

- For example, display only records that have valuable data
- `if` statement checks for valid data
- Output displays in order

## Accessing a random access file randomly

- Benefit of random access files is you can retrieve a specific record directly

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.8 Reading Records from a Random Access Data File (2 of 3)




Figure 11-38: Output of the `ReadEmployeesSequentially` application

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# 11.8 Reading Records from a Random Access Data File (3 of 3)

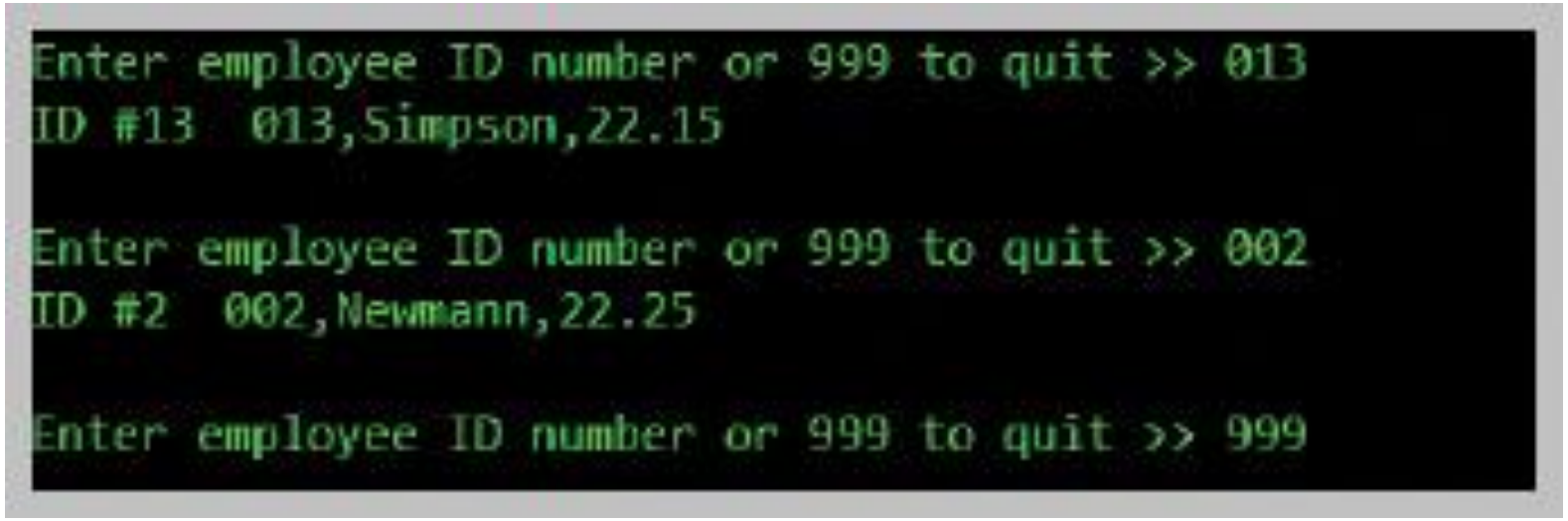



Figure 11-40: Typical execution of the `ReadEmployeesRandomly` application

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# Activity 11.4: Case Study

You have been asked to perform a variety of input and output tasks on a file.

- What does it mean that the `InputStream`, `OutputStream`, and `Reader` classes (which are subclasses of the `Object` class) are abstract?
- Should you include a newline character after a record? Why or why not?
- In a random access file for employees, which of the following would make a good key field: first name, pay rate, or employee ID. Why?
- Can you access a random access file sequentially? Explain your answer.

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# Activity 11.4: Case Study Debrief

- Abstract classes contain methods that must be overridden in their child classes, so the `FileInputStream` subclass of `InputStream` can override the `InputStream` class, for example.
- A data file does not require a newline character after each record, but adding a new line makes the output file easier for a person to read and interpret.
- Many employees might have the same name, or the same rate of pay. An employee ID number is unique to each individual employee. Making it the key field ensures that each record is unique.
- You can process a random access file either sequentially or randomly. You can use an `if` statement when processing a random access file to select only certain records and produce the output in a certain order.

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# Don’t Do It

- Don’t forget a `Path` name might be relative and that you might need to make the `Path` absolute before accessing it.
- Don’t forget that the backslash character starts the escape sequence in Java, so you must use two backslashes in a string that describes a `Path` in the DOS operating system.


Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# Self-Assessment (1)

1. What is the difference between volatile and nonvolatile storage?
2. What is the first step to create a `Path`?
3. Explain the data hierarchy of files, records, fields, and characters.
4. List classes used for input and output.
5. For what purpose would you use the `newline()` method?
6. What is a file channel object used for?
7. Describe how a random access file uses key fields.
8. What is the benefit of using a random access file?

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

# Summary

Click the link to review the objectives for this presentation.

Link to Objectives

CENGAGE

Joyce Farrell, Java Programming, 11th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.